TCP CERL: Congestion Control Enhancement Over Wireless Networks

Implementation-Oriented Presentation

Tasnimul Islam Mahi (2105013)

April 7, 2026

Outline

1. Introduction
2. CERL Mechanism
3. Implementation
4. Performance Evaluation
5. Advantages
6. Implementation Steps
7. Conclusion

Introduction

Problem Statement

TCP Reno suffers severe performance degradation in wireless networks due to **random losses** (non-congestive).

- Reno treats all losses as congestion $\rightarrow$ reduces window unnecessarily
- Throughput drops significantly in lossy environments
- Need to distinguish: **congestion loss** vs **random loss**

Proposed Solution

TCP CERL (Congestion Control Enhancement for Random Loss)
A sender-side modification of TCP Reno that dynamically distinguishes loss types.

Key Idea

- Uses RTT measurements to estimate **bottleneck queue length (l)**
- Sets dynamic threshold $N = A \times l_{max}$
- If $l < N \rightarrow$ treat as random loss (no window reduction)
- If $l \geq N \rightarrow$ treat as congestion loss (reduce window)

Queue Length Estimation

$$l = (RTT - T) \times B$$

- l = queue length
- RTT = round-trip time
- T = minimum observed RTT (propagation + service delay)
- B = bottleneck bandwidth (set to 1 in implementation)

Dynamic Threshold

$$N = A \times l_{max}$$

- l_{max} = maximum observed queue length
- $A = 0.55$ (empirically chosen)

Window Inflation/Deflation

Modifications to Reno:

1. Only reduce window if $I \geq N$ AND first congestion in current window
2. Save current window in `oldcwnd` for later deflation
3. Multiple losses in one window $\rightarrow$ only one reduction

Pseudocode

```
// On receiving 3 duplicate ACKs
if (l >= N && highestAck-1 > lastDecMaxSentSeqno) {
    ssthresh = min(cwnd, awnd) / 2;
    if (ssthresh < 2*segsize)
        ssthresh = 2*segsize;
    cwnd = ssthresh + 3*segsize;
    lastDecMaxSentSeqno = MaxSentSeqno;
} else {
    oldcwnd = cwnd;
    cwnd = cwnd + 3*segsize;
}

// On receiving new ACK
if (oldcwnd > 0) {
    cwnd = oldcwnd;
    oldcwnd = -1.0;
} else {
    cwnd = ssthresh;
}
```

Simulation Setup

Network Topology:

- Mixed wired/wireless
- Bottleneck router with drop-tail queue
- Random loss: 0–10%
- Compared with: Reno, NewReno, Vegas, WestwoodNR, Veno

Throughput Improvement (Single Connection)

TCP Variant	Throughput Gain
TCP Reno	Baseline
TCP NewReno	+153%
TCP Vegas	+85%
TCP WestwoodNR	+64%
TCP Veno	+88%
TCP CERL	**+175%**

Table: Throughput gain at 2% loss rate

Key Insight

CERL outperforms all variants, especially in typical wireless loss range (1–2%)

Throughput vs. Loss Rate

- CERL maintains high throughput up to 10% loss
- Other protocols degrade sharply after 1% loss

Multiple Connections

TCP Variant	Throughput Gain
TCP Reno	Baseline
TCP NewReno	+226%
TCP Vegas	+123%
TCP WestwoodNR	+70%
TCP Veno	+199%
TCP CERL	**+211%**

Table: Multiple connections (coexisting with Reno)

Fairness

CERL competes fairly with Reno when no random loss present. With random loss, it utilizes bandwidth left by degraded Reno connections.

Why CERL is Superior

vs. Veno

- Dynamic threshold (not fixed)
- No window reduction for random loss
- Multiple losses $\rightarrow$ one reduction

General Advantages

- **Simple**: Only sender-side modification
- **Compatible**: Works with existing Reno
- **Deployable**: No network infrastructure changes
- **Robust**: Never worse than Reno

Step-by-Step Implementation

1. **Extend TCP Reno structure**
   - Add variables: `l`, `l_max`, `N`, `oldcwnd`, `lastDecMaxSentSeqno`
2. **Update RTT measurement handler**
   - Calculate queue length l on each RTT update
   - Update l_max and N
3. **Modify fast retransmit/recovery**
   - Integrate loss type decision logic
   - Implement conditional window reduction
4. **Add window inflation/deflation handlers**
   - Manage `oldcwnd` for proper deflation
5. **Testing and validation**
   - Use ns-3
   - Compare with baseline TCP variants

My Proposed Modifications

1. Make A adaptive instead of a hardcoded 0.55

Right now: A=0.55

Better Solution:

Increase A when loss is frequent but RTT is stable

Decrease A when queue oscillations increase

Now CERL learns the network instead of assuming it's polite.

My Proposed Modifications

2. Use the minimum RTT of the last 10 RTT samples rather than a global minimum RTT

A global minimum RTT would be problematic in various scenarios and we want to avoid that. Using the most recent RTT samples would be more reasonable.

My Proposed Modifications

3. Use the maximum bottleneck queue length of the last 20 calculations rather than a global maximum length

Queue length can fluctuate over time, and so, the recent values matter more.

Conclusion

Summary

- TCP CERL significantly improves throughput in lossy wireless networks
- Achieves 175–211% gain over Reno in typical conditions
- Maintains fairness and compatibility
- Simple to implement and deploy

Future Work

- Real-world deployment testing
- Adaptation to modern wireless standards (5G, Wi-Fi 6)
- Integration with other TCP enhancements (BBR, CUBIC)

Ready for implementation!

References

- El-Ocla, H. (2008). TCP CERL: Congestion control enhancement over wireless networks.
- Allman, M., Paxson, V., & Stevens, W. (1999). TCP congestion control.
- Fu, C. P., & Liew, S. C. (2003). TCP Veno: TCP enhancement for wireless.
- Casetti, C., et al. (2002). TCP Westwood.
- Brakmo, L. S., et al. (1994). TCP Vegas.

Thank You! Questions?